

Relatório Prático: O Problema dos Leitores e Escritores

Solução: LeitoSync

Disciplina: Sistemas Distribuídos

Alunos: Gabriel Viana da Costa; Marlon André Pereira Almeida; Pedro Victor Soares da Silva Araújo e Randerson Sousa de Sá Nunes

Professor(a): Francois Fernandes Ribeiro Barbosa

1. Introdução

O LeitoSync é um sistema distribuído voltado ao gerenciamento de leitos e recursos hospitalares críticos, como leitos de UTI, leitos clínicos, salas cirúrgicas, respiradores e equipamentos médicos essenciais.

Em uma rede de saúde moderna, o problema inerente à gestão destes recursos envolve múltiplas unidades hospitalares, operadores, médicos e centrais de regulação consultando e alterando informações compartilhadas sobre recursos críticos de maneira concorrente.

O objetivo principal deste projeto transcende a mera criação de um sistema de operações básicas (CRUD). O projeto foi concebido para demonstrar, na prática e em um cenário distribuído realista, a aplicação do clássico **Problema dos Leitores e Escritores**, provando como arquiteturas modernas resolvem contenção de dados e evitam falhas catastróficas, como a reserva duplicada de um leito crítico de UTI.

2. Objetivo do projeto

Os objetivos principais do LeitoSync são:

- Permitir a consulta rápida e em tempo real de hospitais e de seus respectivos recursos hospitalares;
- Permitir a reserva, ocupação, liberação, bloqueio e manutenção preventiva de recursos;

- Evitar terminantemente a reserva duplicada de um mesmo recurso por centrais distintas operando simultaneamente;
- Permitir múltiplas leituras simultâneas sem gerar penalidade de bloqueio desnecessária;
- Controlar escritas concorrentes sobre o mesmo recurso, preservando a consistência dos dados;
- Registrar uma trilha de auditoria atômica de todas as alterações;
- Demonstrar visual e empiricamente o comportamento por meio de testes automatizados exaustivos e de um simulador frontend interativo.

3. Fundamentação: problema dos leitores e escritores

O problema dos leitores e escritores ocorre quando múltiplos processos ou threads acessam um recurso compartilhado. Neste modelo, vários **leitores** podem acessar o recurso simultaneamente, pois a operação de leitura é idempotente e não altera o estado do sistema. Porém, os **escritores** precisam de acesso estrito e exclusivo (exclusão mútua), pois a operação de escrita modifica o estado compartilhado e pode causar severa inconsistência se ocorrer ao mesmo tempo que outra operação de escrita ou leitura.

Neste escopo, destacam-se os seguintes elementos: * **Leitores:** Entidades que buscam o estado de um recurso. * **Escritores:** Entidades que transacionam e mudam o estado de um recurso. * **Recurso Compartilhado:** A memória ou o registro a ser acessado. * **Região Crítica:** O segmento de código ou transação de banco de dados onde a mutação ocorre. * **Condição de Corrida (Race Condition):** O fenômeno temporal onde o resultado depende da ordem incontrolável de execução das operações. * **Exclusão Mútua:** Garantia de que se um escritor está na região crítica, nenhum outro escritor ou leitor poderá acessá-la. * **Consistência dos Dados:** O reflexo preciso e coerente do domínio após qualquer operação.

A tabela abaixo compara o problema clássico da literatura de Sistemas Distribuídos com a implementação real no LeitoSync:

Problema clássico	LeitoSync
Leitores	Consultas de hospitais, recursos, dashboard e disponibilidade
Escritores	Reservar, ocupar, liberar, bloquear ou colocar recurso em manutenção
Recurso compartilhado	Estado individual dos leitos e recursos hospitalares
Região crítica	Transação de alteração do status e versionamento de um recurso
Risco	Reserva duplicada, estado inconsistente ou perda de atualização
Solução	Transações no PostgreSQL com bloqueio pessimista por linha (SELECT FOR UPDATE)

4. Descrição geral da solução

O LeitoSync adota uma arquitetura em camadas focada em desacoplamento e escalabilidade, empacotada totalmente em containers para portabilidade e reprodutibilidade.

Tecnologias Utilizadas:

- **Backend:** Python, FastAPI, SQLAlchemy assíncrono, PostgreSQL, Alembic (migrações), Pytest (testes concorrentes), e WebSockets nativos.
- **Frontend:** React, TypeScript, Vite, e Tailwind CSS.
- **Infraestrutura:** Docker e Docker Compose isolando backend, frontend e banco de dados.

Responsabilidade de Cada Camada:

- **Frontend:** Responsável por prover a interface visual de alta responsividade. Consome as APIs REST, exibe o dashboard, gerencia a tela de detalhes do recurso, provê a tela

de auditoria, além de abrigar o núcleo didático do MVP: o Simulador de Leitores e Escritores.

- **Backend:** Processa a lógica da API REST, aplica as regras de negócio de transição de estado, gerencia o controle de concorrência com o ORM, injeta logs de auditoria de forma atômica e emite eventos de WebSocket (`RESOURCE_UPDATED`).
- **Banco de Dados (PostgreSQL):** Responsável primário pelo armazenamento persistente, integridade relacional, e mais importante: o controle transacional e bloqueio pessimista a nível de linha.
- **Testes:** Validação profunda das garantias de concorrência e consistência simulando cenários idênticos à produção.

5. Modelagem do domínio

O sistema foi modelado de forma enxuta para validar a premissa de distribuição. As principais entidades mapeadas nos arquivos (`backend/app/models/`) são:

- **User:** Tabela de suporte para identificar quem solicita operações (não utilizada fortemente em prol de MVP, onde focou-se nos metadados da requisição `actor_name`).
- **Hospital:** Representa as unidades de saúde distribuídas geograficamente.
- **ResourceType:** Categoria de um recurso (Ex: “Leito de UTI”, “Respirador Mecânico”).
- **Resource:** A entidade núcleo, o *recurso compartilhado*. Ela liga o recurso a um Hospital e a um ResourceType.
- **Reservation:** Histórico ou ticket das intenções ativas ou passadas de manter a posse momentânea de um Resource.
- **AuditLog:** Log apensado imutável (append-only) atrelado estritamente à transação do Resource para manter a trilha do que aconteceu e quem requisitou.
- **ConcurrencyEvent:** Tabela complementar que pode abrigar observabilidade (métricas) e alertas de concorrência rejeitada.

Foco na entidade Resource: Cada recurso (ex: o leito UTI-A-102) abriga duas colunas essenciais: `status` e `version`. Os principais status possíveis para o domínio são: * **available:** Livre para reserva. * **reserved:** Intenção de uso recém consolidada; recurso não

disponível. * **occupied:** Paciente admitido fisicamente ou equipamento em uso. * **blocked:** Interditado temporariamente (limpeza). * **maintenance:** Recurso indisponível por razões técnicas preventivas.

6. Operações de leitura

Diversas operações do LeitoSync não exigem exclusão mútua e se comportam estritamente como **leitores** no modelo teórico.

Endpoints como: * GET /resources * GET /resources/{id} * GET /resources/availability * GET /hospitals * GET /resource-types * GET /dashboard/summary * GET /audit-logs

Essas rotas executam **select** diretos através do repositório, sem requerer cláusulas de lock na base. Eles representam o lado “Read-Heavy” de um sistema de regulação de leitos.

No LeitoSync, as consultas de disponibilidade representam os leitores perfeitamente: uma central de regulação e cinco hospitais podem consultar simultaneamente a disponibilidade de vagas sem causar nenhum gargalo de travamento uns nos outros, extraindo os metadados do estado de maneira livre (non-blocking).

7. Operações de escrita

As requisições que promovem a mudança de estado num leito ou respirador são os **escritores**.

Endpoints como: * POST /resources/{resource_id}/reserve * POST /resources/{resource_id}/occupy * POST /resources/{resource_id}/block * POST /resources/{resource_id}/maintenance

Essas operações manipulam o estado e incrementam a versão do recurso, necessitando de uma coordenação drástica (bloqueio). Sem sincronização, a consistência do modelo quebra. Exemplos práticos dessa inconsistência (condições de corrida) caso a exclusão mútua não fosse resolvida:

- Duas centrais, ao enxergarem o leito UTI-01 como “available” na tela, clicam em

“Reservar” ao mesmo tempo. Ambas requisições são processadas paralelamente na camada de aplicação e despacham dois `UPDATE` confirmando a reserva. Resultado: A falha mais temida — duas ambulâncias enviadas para um mesmo leito.

- Um recurso sendo liberado por um profissional ao mesmo tempo em que outro decide ocupá-lo emergencialmente, sobrescrevendo um estado por outro baseado em leitura desatualizada.
- Histórico de auditoria incompleto ou inconsistente devido a salvamentos concorrentes na tabela base (dirty reads).

8. Como o código soluciona o problema

Para resolver essa condição de corrida clássica em ambientes distribuídos, o sistema descentraliza a contenção que estaria apenas na memória (FastAPI) e a delega ao Banco de Dados (PostgreSQL).

A solução reside, por excelência, no arquivo `backend/app/repositories/resource_repo.py`, na camada mais íntima do acesso ao dado.

Fluxo atômico implementado para Reserva: 1. O cliente envia uma requisição via API (`/reserve`) para o backend. 2. O SQLAlchemy é instruído a abrir uma transação atômica vinculada a sessão (`begin()`). 3. O recurso é buscado no banco através do método `with_for_update()` (bloqueio pessimista de linha `SELECT FOR UPDATE`). 4. Em memória restrita (travada para transações paralelas), o sistema verifica as premissas (ex: se o status ainda é `available`). 5. Se aprovado, o status na memória é alterado para `reserved`. 6. O metadado de `version` sofre autoincremento (`v1 -> v2`). 7. Uma entidade de `Reservation` ativa é criada na mesma sessão. 8. Uma entidade de `AuditLog` é criada na mesma sessão, rastreando a mudança (`old -> new`). 9. Ocorre o `commit()` das alterações, encerrando o tempo de vida da exclusão mútua na região crítica e persistindo fisicamente os 3 comportamentos acima num único movimento (All or Nothing). 10. O FastAPI emite no WebSocket `/ws/resources` o broadcast confirmando a mudança a todos os visualizadores. 11. Se um **segundo escritor** estivesse aguardando o lock da mesma linha neste milissegundo e tentasse reservar ao receber o lock, ele encontraria o status como `reserved` no passo 4. Imediatamente a transação dele sofre

rollback() no banco e o backend o avisa retornando amigavelmente HTTP 409 Conflict.

Pseudocódigo do Fluxo de Banco:

```
BEGIN TRANSACTION

SELECT recurso
FROM resources
WHERE id = resource_id
FOR UPDATE

IF recurso.status != available:
    ROLLBACK
    RETURN 409 Conflict

UPDATE resources
SET status = reserved,
    version = version + 1

INSERT INTO reservations (...)
INSERT INTO audit_logs (...)

COMMIT
```

Essa poderosa funcionalidade previne, por design arquitetural, o duplo agendamento de recursos críticos de saúde independentemente do número de requisições por segundo.

9. Tratamento de conflitos

O retorno HTTP semântico implementado nas rotas para rejeição concorrente é o HTTP 409 Conflict.

Quando um recurso não está mais disponível para reserva devido a um escritor

concorrente que chegou instantes antes, não seria correto retornar HTTP 400 (Bad Request), visto que o pedido inicialmente era sintaticamente correto. Assim, o erro 409 relata o esbarro de estado do sistema distribuído.

Exemplo Prático na vida real (LeitoSync): Dois operadores, da Central Leste e da Central Sul, visualizam o leito de respirador RESP-05 como disponível na tela de seus navegadores. Ambos pressionam “Reservar” de maneira simultânea no sistema. Ambas as conexões assíncronas batem no **FastAPI** em paralelo. A transação inicial da Central Sul é mais rápida em obter o **SELECT FOR UPDATE** da linha 5 no Postgres. A Central Leste aguarda, bloqueada, a liberação daquela row. A Central Sul adquire o bloqueio, muda para “reservado”, audita a alteração e comita os dados. Quando o banco libera a trava e a transação da Central Leste lê a linha (agora atualizada), descobre no Python que o status não é mais “available”. Imediatamente o sistema levanta um **ConcurrencyConflictException**, que por via da camada de **routes** do FastAPI devolve um código 409 Conflict ao operador da Central Leste informando “*Resource is no longer available*”, provando que o sistema é totalmente robusto.

10. Auditoria

Para manter conformidade rígida de sistemas de saúde, foi mapeada e preenchida a entidade **AuditLog**.

Toda operação mutacional no repositório preenche esta tabela dentro da própria transação do SQLAlchemy, injetando dados como: * O ID do recurso; * Ação engatilhada (ex: “RESERVE”, “RELEASE”); * Status obsoleto (old_status) e novo status (new_status); * Ator requisitante (actor_name) para responsabilização legal; * Timestamps automáticos (data/hora).

O valor do design atômico está na sua imutabilidade e rastreabilidade fidedigna: caso uma transação exploda e sofra falha na comunicação de rede que faria um rollback da alteração, o Audit Log **também retrocede**. Jamais existirá um Audit Log orfão de um estado de recurso. Isso provê confiança absoluta, permite investigações fáceis por administradores e exprime transparência irrestrita do gerenciamento de vagas à Secretaria da Saúde e aos

Hospitais.

11. Atualização em tempo real

Para lidar dinamicamente com sistemas reativos, o LeitoSync conta com um serviço nativo interno de broadcast via WebSocket atachado à rota `ws://localhost:8000/ws/resources`.

Sempre que a API realiza um commit de mutação de recurso, a dependência conectada engatilha um evento JSON às telas dos hospitais conectados. É crucial entender que **o WebSocket NÃO é o mecanismo que garante consistência no projeto**. A exclusão mútua e concorrência habitam e se definem na segurança pessimista do Banco de Dados. O WebSocket age exclusivamente de maneira passiva, propagando apenas a notificação final de mutação para a interface visual das demais centrais, eliminando a pesada técnica de recarregamento manual da tela (polling).

12. Simulador leitores-escritores

Um dos grandes expoentes do LeitoSync, construído no frontend, é a aba visual do **Simulador**.

Ele foi projetado estritamente como um laboratório de provas com o intuito de confirmar empiricamente o conceito da disciplina de Sistemas Distribuídos e as garantias da arquitetura. O simulador permite apontar para um Recurso livre (ex: um leito disponível), determinar simultaneamente N Leitores (ex: 20 requisições disparadas paralelamente via Promise.all) e N Escritores (ex: 5 requisições disparadas no exato mesmo frame).

Resultados demonstrados pelo Simulador: * Múltiplos leitores podem consultar o leito sem sofrerem penalidades ou atrasos, exibindo centenas de respostas 200 OK. * Múltiplos escritores em concorrência predatória acionam perfeitamente o Lock. * É retornado com êxito **apenas e singularmente uma única escrita confirmada (200)** que deteve o lock primário; e dezenas de escritas imediatamente rejeitadas com erro 409 impedindo double-bookings e vazamento de vagas, provando visivelmente a aplicabilidade teórica da literatura na vida real da rede hospitalar.

13. Testes automatizados

Uma suíte rigorosa de baterias end-to-end e integração assíncrona foi redigida em `backend/tests/test_resources.py` com o framework Pytest, totalizando **8 testes automatizados**, todos aprovados com êxito em isolamento (`TRUNCATE` tables).

1. `test_get_resources`: Valida a camada de leitura básica garantindo integridade de retorno da coleção;
2. `test_reserve_available_resource`: Certifica que o caso ideal mutacional de uma reserva altera o estado com sucesso para HTTP 200;
3. `test_reserve_already_reserved_resource`: Assegura a barreira não-concorrente sequencial; tenta reservar o já reservado e obtém obrigatoriamente um erro semântico HTTP 409;
4. `test_concurrent_reservations`: **O Teste de Fogo (Escritores)**. Usa `asyncio.gather` para simular 5 chamadas no exato microsegundo sobre o mesmo leito, assegurando e asserindo que exatos 4 deles retornem obrigatoriamente erro 409 e apenas 1 obtenha 200 OK.
5. `test_concurrent_readers`: Dispara **20 leituras simultâneas** de status sobre o recurso. Confirma que os 20 requests têm vida ilimitada, asserindo 200 OK em todos, e que a leitura, de fato, é não-violenta e idempotente para a consistência da API.
6. `test_readers_during_writer_attempt`: Associa o agrupamento de **20 Leitores com 1 Escritor concorrendo no exato mesmo ms**. Afere que a escrita domina a região e se consolida, enquanto as 20 leituras conseguem escorregar fora da exclusão e reportarem sem delays que conseguiram dados com 200 OK sem engasgos.
7. `test_reservation_creates_single_active_reservation`: Bate diretamente de maneira violenta sobre a integridade do ORM e Banco. Dispara o enxame de 5 escritores e consome o comando direto (`func.count()`) na base do PostgreSQL para aferir, cabalmente, se somente 1 linha ativa de Reservação fora gerada para as 5 requisições, extirpando hipóteses de ghost-bookings.
8. `test_write_creates_audit_log`: Garante que, independente do que suceda na arquitetura, toda solicitação bem sucedida deve carregar a tiracolo a atomicidade da trilha

de segurança do LeitoSync, aferindo status antigos (old) e novos (new).

(Os testes foram validados e rodaram no container backend apontando 8 passed e zero falhas de vazamento).

14. Execução do projeto

O projeto baseia-se inteiramente no Docker. Para executá-lo, baixe a pasta contendo a arquitetura:

1. Na raiz do projeto, acione o comando para iniciar e atrelar instâncias do ambiente com orquestração automática:

```
docker compose up --build
```

2. Acesse as instâncias vitais através dos portais no host base:

- **Interface Frontend interativa (React):** <http://localhost:5173>
- **Motor da API JSON (FastAPI):** <http://localhost:8000>
- **Documentação de uso e schemas abertos da API (Swagger):**
<http://localhost:8000/docs>

3. **Para Rodar as baterias de Suíte de Validação Pytest:** Com os servidores já de pé operando, abra outra janela no seu terminal na raiz do projeto e envie o engatilho para o runner de testes dentro do ambiente isolado do backend Python:

```
docker compose exec backend pytest -v
```

*(Nota de Rede Interna Docker: A API consome ativamente o PostgreSQL mapeado no node network host interno da engine chamado **db:5432**).*

15. Limitações do projeto

Sendo formatado explicitamente para demonstrações robustas de prova-de-conceito (MVP) para domínios de Sistemas Distribuídos e garantias transacionais de PostgreSQL em ambiente concorrente, é compreensível e tolerável a ausência das seguintes engrenagens complexas de enterprise:

- Não existe motor real de segregação e encriptação com Autenticação (JWT, Sessões Hash). O campo `actor_name` injetável faz as vezes de identificação simplificada no design.
- O controle em ambiente de permissões hierárquicas (RBAC) não existe, onde em cenários ideais apenas perfis de “Regulação Governamental” sobreporiam ações em relação a perfis de “Médicos Locais”.
- O modelo propaga dados ao Frontend reativo com uma única instância nativa WebSocket do FastAPI. Em um sistema horizontal global da vida real, um cluster necessitaria do backplane com um cache in-memory escalável (ex: *Redis*) agindo como Publisher/Subscriber entre as inúmeras cópias espalhadas das instâncias de backend.
- O PostgreSQL centraliza sozinho toda a carga (ponto central de falha e de consistência); não foram modeladas camadas pesadas de consenso distribuído externo ao banco ou multi-master sharding geográfico das linhas.

Estas restrições mantêm o foco irrestrito na simplicidade acadêmica, permitindo vislumbrar o real motivo e propósito deste case que é comprovar o manuseio assíncrono e concorrente (readers e writers constraints).

16. Possíveis melhorias futuras

Com vistas ao futuro ou escopos avançados e extensões deste projeto de código aberto, sugere-se os seguintes passos na esteira arquitetônica:

- Implementar Autenticação OAuth2 + controle rígido de autorização por papéis (Role-Based Access Control).
- Adição do Broker assíncrono Redis para garantir emissões e retentativas do WebSocket perfeitamente para múltiplas réplicas do backend.
- Integrações de monitoramento APM massivo, instrumentando o código em containers nativos do Prometheus, aliado a painéis em Grafana para demonstrar, nos logs, as rejeições contínuas HTTP 409 da engine para controle analítico dos tempos operacionais de reserva da UTI.
- Implementação de política de `timeouts`: As reservas deveriam decair para `Available`

em 30 minutos caso o hospital não mude o registro de admissão interna para `Occupied`.

- Mapeamento de Fila de Prioridade Baseada em Triage para urgências críticas.

17. Conclusão

O **LeitoSync** comprova e endossa com notório primor a aplicação do problema clássico dos leitores e escritores na modelagem vital de um Sistema Distribuído para Saúde, em especial onde falhas lógicas e de transações assíncronas geram efeitos mortais e prejuízos físicos, não meramente corrompimentos numéricos (como em bancos financeiros).

O sistema permitiu escalabilidade horizontal massiva (múltiplas leituras paralelas de disponibilidade ocorrendo simultaneamente sem restrições em sua arquitetura livre) protegendo as escritas concorrentes em trincheiras seguras por intermédio das garantias de integridade do Banco de Dados via bloqueio pessimista relacional (`SELECT FOR UPDATE` sob o ORM `SQLAlchemy`).

As validações automatizadas end-to-end — especialmente a validação por agrupamentos do `asyncio.gather` injetando microcolisões nos endpoints concorrentes — somadas ao simulador gráfico Frontend fecham o cerco confirmando que tentativas violentas de reserva duplicada num microssegundo reagem no funil adequado e inevitavelmente terminam numa única transação garantida e auditável de estado consistente, rechaçando instantaneamente anomalias de double-booking com status `HTTP 409 Conflict`. A robustez apresentada confirma o propósito do projeto de servir à disciplina e pregar a eficácia real, teórica e programática de coordenação em modelos distribuídos.